

BiteBuddy Complete Reverse Engineering & Viva Preparation Guide

This architectural handbook is a complete reverse-engineering and line-by-line review of the BiteBuddy project. This guide is organized into 15 specific modules containing design patterns, source code walkthroughs, data flows, and 100 potential viva/interview questions.

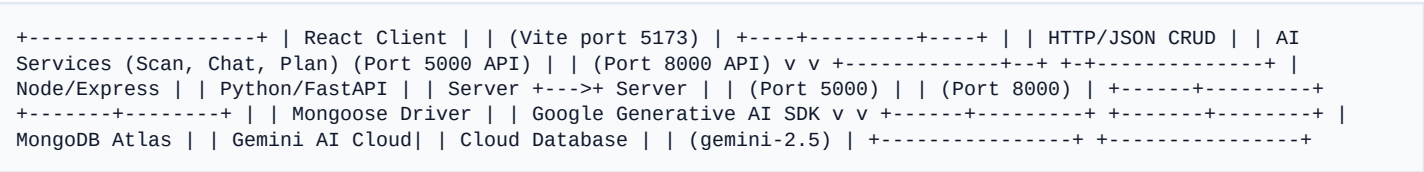
PART 1 — PROJECT OVERVIEW

1. Overall Architecture

BiteBuddy uses a Dual-Backend Microservices Architecture coupled with a Component-Driven Single Page Application (SPA) frontend:

- 1. Frontend (Client Layer): React + Vite (serving on port 5173). It manages the interactive dashboard, food scanner upload previews, dynamic meal calendar grid, shopping lists, and real-time chat interface.
- 2. Core Backend (Business Logic & CRUD Layer): Node.js + Express (serving on port 5000). It manages database interactions with MongoDB Atlas, CRUD operations for profiles, logs, and workout metrics, and serves uploaded static assets.
- 3. AI Backend (Stateless Intelligence Layer): Python + FastAPI (serving on port 8000). It interfaces directly with Google's Generative AI SDK, running multimodal vision analysis, text generation, and JSON schema extraction.
- 4. Database (Data Persistence Layer): MongoDB Atlas. It stores schema-enforced, JSON-like document models for user stats, workouts, meals, and AI plans.

Architectural Flow Chart



2. Tech Stack Selection Rationale

Component	Technology Stack	Rationale
Frontend	React (Vite)	Modular component architecture, quick Hot Module Replacement (HMR) for visual styling, declarative UI paradigm, and virtual DOM diffing for high-performance dashboard renders.
Backend 1	Node.js + Express	Single-threaded asynchronous non-blocking event-driven loop handles high concurrency of read-write operations for CRUD features (water trackers, weight entry, logs) with zero latency.

Backend 2	Python + FastAPI	Native async capabilities, auto OpenAPI documentation, and Python's seamless integration with the Google Generative AI SDK. FastAPI handles heavy data serialization/deserialization with Pydantic.
Database	MongoDB (Mongoose)	High flexibility for polymorphic structures like dynamic meal plans (which vary based on user diet types) and shopping list arrays. Schema validation is handled easily at the application level via Mongoose.
AI Model	Gemini-2.5-Flash	Multimodal inputs (processing food photos directly), high-speed token generation (low latency for chat updates), and capability to output schema-conforming JSON structure directly.

3. Folder Structure Diagram

```
ai-learning/
├── backend/
│   ├── config/
│   │   └── db.js <-- MongoDB Atlas connection & life-cycle listeners
│   ├── controllers/
│   │   ├── goal.controller.js <-- Goal CRUD operations
│   │   ├── mealController.js <-- Multer image upload & Gemini image scan bridge
│   │   ├── mealplan.controller.js <-- Save/Retrieve manual planner adjustments
│   │   └── shopping.controller.js <-- Manage shopping list checkboxes and items
│   ├── models/
│   │   ├── MealLog.js <-- Consumed meals schema
│   │   ├── MealPlan.js <-- AI generated 7-day plan schema
│   │   ├── ShoppingList.js <-- Grocery check items schema
│   │   ├── UserProfile.js <-- Onboarding demographic details schema
│   │   ├── WorkoutLog.js <-- Logged exercises schema
│   │   └── goals.js <-- User target goals schema
│   ├── routes/
│   │   ├── generate.js <-- Express-FastAPI gateway for personalized plans
│   │   ├── goals.js <-- Express goal routes
│   │   ├── meallog.js <-- Express meal log routes
│   │   ├── mealplan.js <-- Express meal plan database route
│   │   ├── meals.js <-- Multer multi-part configuration route
│   │   ├── profile.js <-- Express profile onboarding route
│   │   ├── shopping.js <-- Express shopping list routes
│   │   └── workouts.js <-- Express workout routes
│   ├── services/
│   │   └── geminiService.js <-- Node-side backup Gemini client implementation
│   ├── uploads/ <-- Statically served storage directory for food images
│   ├── .env <-- Connection strings & secrets
│   ├── app.js <-- Express application configuration entrypoint
│   ├── main.py <-- FastAPI server entrypoint
│   ├── meal_scan.py <-- FastAPI image analysis route
│   ├── ai_personalize.py <-- FastAPI profile-to-weekly-meal-plan pipeline
│   ├── shopping_ai.py <-- FastAPI weekly-plan-to-groceries pipeline
│   └── prompts.py <-- System prompts for nutrition extraction & chat personality
├── requirement.txt <-- Python package dependencies
├── frontend/
│   ├── src/
│   │   ├── assets/ <-- Local UI mock images and SVG assets
│   │   ├── components/
│   │   │   ├── charts/
│   │   │   │   ├── MacroRing.jsx <-- Progress circles for protein/fat/carbs/calories
│   │   │   │   └── WeeklyChart.jsx <-- Calorie bars showing historical weekly trend
│   │   │   ├── MobileNav.jsx <-- Tab bar for responsive viewports
│   │   │   └── Sidebar.jsx <-- Navigation sidebar
│   │   ├── ui/ <-- Cards, Buttons, Inputs, Modals, Progress bars
│   │   ├── data/
│   │   ├── constants.js <-- Hardcoded prices, food items, search links
│   │   ├── hooks/
│   │   │   └── useToast.jsx <-- Custom React Context hooks for toast notifications
│   │   ├── pages/
│   │   │   ├── AICoach.jsx <-- Chat window with the Gemini AI engine
│   │   │   ├── Dashboard.jsx <-- Bento-grid layout with calorie counts and hydrations
│   │   │   ├── FoodLog.jsx <-- Grouped list of breakfast, lunch, and dinner logs
│   │   │   ├── FoodScanner.jsx <-- File upload / Camera trigger for food detection
│   │   │   ├── Goals.jsx <-- Target settings panel
│   │   │   ├── MealPlanner.jsx <-- 7x3 grid containing diet recipes
│   │   │   ├── Onboarding.jsx <-- Step-by-step profile form
│   │   │   ├── Settings.jsx <-- Profile editing, Dark mode theme toggle
│   │   │   ├── Shopping.jsx <-- Interactive checklist and Q-Commerce redirect
│   │   │   └── WorkoutLog.jsx <-- Workouts logs form
│   │   └── App.jsx <-- Root router & profile loader
│   └── main.jsx <-- Dom renderer entrypoint
```

4. Responsibilities of Each Layer

- React (Frontend): Responsible for capturing user interactions, rendering progress bars, displaying responsive UI modals, managing page routes, and uploading files using FormData payloads.
- Express (Node.js): Responsible for serving static files, handling database writes and reads for user profiles, workouts, and plans, managing image storage location routing using Multer, and routing calls to the FastAPI server.
- FastAPI (Python): Acts as a dedicated AI Gateway. It handles asynchronous integration with the Gemini API, processes multi-part image streams directly, enforces strict Pydantic model schemas, and parses raw text responses from Gemini into JSON.

- MongoDB: Stores demographic profiles, logs, and workout metrics. It uses compound indexes (such as user ID and diet type) to ensure efficient retrieval of target documents.
 - Gemini AI: Functions as the primary intelligence backend. It detects food items in images, calculates nutritional estimates (calories, protein, carbs, fats), generates personalized weekly recipes based on user profiles and workouts, and acts as a conversational fitness coach.
-

PART 2 — FOLDER WALKTHROUGH

1. Backend Folders

- **backend/config/**: Sets up Mongoose and connects to MongoDB Atlas. It handles reconnect attempts if the connection drops and runs a graceful shutdown function if the Node process receives a termination signal (SIGINT).
- **backend/controllers/**: Isolates business logic from request endpoints. It processes database queries, executes upsert queries for data persistence, and validates API input formats.
- **backend/models/**: Defines the data models and constraints for our database using Mongoose schemas.
- **backend/routes/**: Handles incoming API requests, runs Multer middleware for file uploads, and forwards data between Express and FastAPI.
- **backend/services/**: Holds modular external services. Includes geminiService.js for executing Node-level calls to the Google Generative AI client.
- **backend/uploads/**: Serves as a local storage directory for uploaded files. The Express server uses static directory middleware to serve these files directly via HTTP links.

2. Frontend Folders

- **frontend/src/components/layout/**: Houses global UI layouts, including the Sidebar for desktop navigation and MobileNav for compact mobile viewports.
 - **frontend/src/components/charts/**: Standardizes visualization charts. Includes MacroRing (renders nested SVG circular bars for macronutrients) and WeeklyChart (draws vertical bars comparing actual vs. target calorie intakes).
 - **frontend/src/components/ui/**: A library of reusable presentational components (Buttons, Inputs, Modals, ProgressBars, Skeletons, and Toast notifications) to maintain design consistency.
 - **frontend/src/data/**: Manages static configuration values, including default item pricing, Quick-Commerce search links, and conversation suggestions.
 - **frontend/src/hooks/**: Exposes reusable React custom hooks, such as useToast, which provides access to the global notification container.
 - **frontend/src/pages/**: Holds top-level page views (such as the Dashboard, Onboarding, FoodScanner, Shopping page, and AI Coach interface) that handle data fetching and coordinate child components.
-

PART 3 — EVERY FILE IN DETAIL

Here is an analysis of each code module in the repository:

1. backend/app.js

- Purpose: Server setup and entry point.
- Why it exists: Configures Express, mounts middleware, imports routes, establishes the database connection, and starts the HTTP server.
- Who calls it: Node.js runtime (node app.js).
- Exports: None. Sets up and runs the application server.
- Data Flow: Directs incoming requests to the correct route handler based on their URL paths (/api/meals, /api/profile, etc.).
- Interactions: Runs config/db.js to connect to MongoDB, sets up static file serving for uploads/, and registers all router endpoints.
- Dependents: package.json start scripts.
- Architecture Fit: Server entry point.

2. backend/config/db.js

- Purpose: Database configuration.
- Why it exists: Manages connection states and retry logic for MongoDB.
- Who calls it: Loaded by app.js during server startup.
- Exports: Configured Mongoose instance.
- Data Flow: Connects to the database string defined in the environment variables (MONGODB_URI).
- Interactions: Interfaces with MongoDB Atlas using the Mongoose ODM driver.
- Dependents: app.js (requires it to establish connection).
- Architecture Fit: Connection lifecycle manager.

3. backend/controllers/goal.controller.js

- Purpose: Goals controller.
- Why it exists: Handles database logic for creating, updating, and retrieving user goal targets.
- Who calls it: backend/routes/goals.js.
- Exports: saveGoal (finds and updates goals for the active user profile) and getGoal (fetches the latest goal configurations).
- Data Flow: Saves goal targets (diet types, calorie requirements, weights) to the Goal model.
- Interactions: Queries the goals collection in MongoDB.
- Dependents: backend/routes/goals.js.
- Architecture Fit: Goal management logic.

4. backend/controllers/mealController.js

- Purpose: Meal controller.
- Why it exists: Handles meal photo scans by extracting file names, sending them to the Gemini API, and saving the results.
- Who calls it: backend/routes/meals.js.
- Exports: scanMeal.
- Data Flow: Reads file paths from Multer (req.file.path), calls the geminiService.js parser, saves the returned macros, and returns the saved document.

- Interactions: Interacts with geminiService.js and the MealLog model.
- Dependents: backend/routes/meals.js.
- Architecture Fit: Image processing controller.

5. backend/controllers/mealplan.controller.js

- Purpose: Meal plan manager.
- Why it exists: Allows users to save customized weekly meal plans.
- Who calls it: backend/routes/mealplan.js.
- Exports: saveMealPlan and getMealPlan.
- Data Flow: Checks request body parameters (diet, plan) and runs upsert queries based on user identifiers.
- Interactions: Updates documents in the MealPlan collection.
- Dependents: backend/routes/mealplan.js.
- Architecture Fit: Planner storage logic.

6. backend/controllers/shopping.controller.js

- Purpose: Shopping list manager.
- Why it exists: Saves state updates for user grocery items.
- Who calls it: backend/routes/shopping.js.
- Exports: saveShoppingList and getShoppingList.
- Data Flow: Validates item formats and saves list data using upsert queries.
- Interactions: Reads and writes to the ShoppingList collection.
- Dependents: backend/routes/shopping.js.
- Architecture Fit: Shopping persistence controller.

7. backend/models/MealLog.js

- Purpose: Meal log schema.
- Why it exists: Defines the database schema for logged meals (calories, protein, carbs, fats, image references, and timestamps).
- Who calls it: Mongoose.
- Exports: MealLog Mongoose Model.
- Interactions: Stores records in the meallogs collection.
- Dependents: Controllers (mealController.js) and routes (routes/meallog.js).
- Architecture Fit: Data schema definition.

8. backend/models/MealPlan.js

- Purpose: Meal plan schema.
- Why it exists: Stores AI-generated weekly plans, target goals, recovery insights, and shopping lists.
- Who calls it: Mongoose.

- Exports: MealPlan Mongoose Model.
- Interactions: References the UserProfile schema.
- Dependents: routes/generate.js and controllers/mealplan.controller.js.
- Architecture Fit: Plan storage model.

9. backend/models/ShoppingList.js

- Purpose: Shopping list schema.
- Why it exists: Standardizes how grocery items are saved, using a compound index to restrict users to one list per diet type.
- Who calls it: Mongoose.
- Exports: ShoppingList Mongoose Model.
- Interactions: Defines array structures matching shoppingItemSchema.
- Dependents: controllers/shopping.controller.js.
- Architecture Fit: Shopping list schema.

10. backend/models/UserProfile.js

- Purpose: User profile schema.
- Why it exists: Stores onboarding parameters, including allergies, weight metrics, target goals, activity levels, and water goals.
- Who calls it: Mongoose.
- Exports: UserProfile Mongoose Model.
- Interactions: Serves as the primary reference for workout logs, meal plans, and goal calculations.
- Dependents: routes/profile.js, routes/generate.js, and routes/meallog.js.
- Architecture Fit: Core profile schema.

11. backend/models/WorkoutLog.js

- Purpose: Workout log schema.
- Why it exists: Defines database schemas for logged workouts (exercises, durations, and calories burned).
- Who calls it: Mongoose.
- Exports: WorkoutLog Mongoose Model.
- Interactions: Stores records in the workoutlogs collection.
- Dependents: routes/workouts.js and routes/generate.js.
- Architecture Fit: Workout data schema.

12. backend/models/goals.js

- Purpose: Goal schema.
- Why it exists: Defines database schemas for target goal metrics (diet types, weights, and daily calories).
- Who calls it: Mongoose.
- Exports: Goal Mongoose Model.

- Interactions: Saves target settings.
- Dependents: controllers/goal.controller.js.
- Architecture Fit: Target metrics schema.

13. backend/routes/generate.js

- Purpose: AI generation gateway route.
- Why it exists: Orchestrates generating a weekly meal plan by querying the user profile and workout history, sending it to FastAPI, saving the response to the database, and returning it.
- Who calls it: React client (/api/generate).
- Exports: Express Router instance.
- Data Flow: Fetches the latest UserProfile and WorkoutLog documents, sends them to <http://localhost:8000/api/ai/personalize> using the fetch API, saves the response, and returns the result.
- Interactions: Connects Express to the FastAPI server and manages MongoDB updates for meal plans.
- Dependents: app.js.
- Architecture Fit: Cross-server communication layer.

14. backend/routes/goals.js

- Purpose: Goal routing.
- Why it exists: Maps goal endpoints to their controller handlers.
- Who calls it: React client (/api/goals).
- Exports: Express Router instance.
- Interactions: Routes requests to the goal.controller.js file.
- Dependents: app.js.
- Architecture Fit: Routing layer.

15. backend/routes/meallog.js

- Purpose: Meal log router.
- Why it exists: Implements CRUD endpoints for logging daily meals.
- Who calls it: React client (/api/meallog).
- Exports: Express Router.
- Interactions: Reads user profiles, creates meal logs, and runs updates based on URL parameters.
- Dependents: app.js.
- Architecture Fit: CRUD routing layer.

16. backend/routes/mealplan.js

- Purpose: Meal plan router.
- Why it exists: Exposes endpoints for saving and loading active meal plans.
- Who calls it: React client (/api/mealplan).

- Exports: Express Router.
- Interactions: Routes requests to controllers/mealplan.controller.js.
- Dependents: app.js.
- Architecture Fit: Routing layer.

17. backend/routes/meals.js

- Purpose: Scan router.
- Why it exists: Configures Multer storage engines and handles file upload requests on the Express server.
- Who calls it: React client (/api/meals/scan).
- Exports: Express Router.
- Interactions: Runs Multer disk-storage middleware and forwards the uploaded image to the scanMeal controller.
- Dependents: app.js.
- Architecture Fit: File upload router.

18. backend/routes/profile.js

- Purpose: Profile router.
- Why it exists: Exposes endpoints for reading, saving, and updating user profiles.
- Who calls it: React client (/api/profile).
- Exports: Express Router.
- Interactions: Reads and updates the UserProfile collection.
- Dependents: app.js.
- Architecture Fit: CRUD routing layer.

19. backend/routes/shopping.js

- Purpose: Shopping router.
- Why it exists: Exposes endpoints for managing shopping items.
- Who calls it: React client (/api/shopping).
- Exports: Express Router.
- Interactions: Routes requests to controllers/shopping.controller.js.
- Dependents: app.js.
- Architecture Fit: Routing layer.

20. backend/routes/workouts.js

- Purpose: Workout router.
- Why it exists: Exposes endpoints for logging and fetching workout histories.
- Who calls it: React client (/api/workouts).
- Exports: Express Router.
- Interactions: Performs query selections on the WorkoutLog collection.

- Dependents: app.js.
- Architecture Fit: CRUD routing layer.

21. backend/services/geminiService.js

- Purpose: Node-level Gemini client.
- Why it exists: Provides an alternate or backup method to call the Gemini API directly from Node.js using base64 image strings.
- Who calls it: backend/controllers/mealController.js.
- Exports: analyzeImage(imagePath).
- Data Flow: Reads local images, converts them to base64 strings, sends them to Gemini using the Node.js SDK, parses the returned text as JSON, and returns the parsed object.
- Interactions: Reads the local filesystem and calls the Google Generative AI endpoint.
- Dependents: controllers/mealController.js.
- Architecture Fit: Node integration service.

22. backend/main.py

- Purpose: FastAPI server entry point.
- Why it exists: Initializes the FastAPI application, configures CORS, initializes the Google Generative AI client using the API key, and mounts python routers.
- Who calls it: Uvicorn (uvicorn main:app --reload).
- Exports: FastAPI App Instance (app).
- Interactions: Loads system prompts and registers routes from meal_scan.py, shopping_ai.py, and ai_personalize.py.
- Dependents: Python server startup execution.
- Architecture Fit: Python backend entry point.

23. backend/meal_scan.py

- Purpose: Python-level image scanner.
- Why it exists: Exposes the /api/meals/scan endpoint on port 8000, which handles image analysis, calls the Gemini model, and returns nutrition estimates.
- Who calls it: React Client (FoodScanner.jsx).
- Exports: APIRouter instance (router).
- Data Flow: Receives image files, saves them locally, reads their byte streams, calls Gemini, parses the returned JSON text block, and returns the parsed metrics.
- Interactions: Interacts with the FOOD_ANALYSIS_PROMPT prompt configurations.
- Dependents: main.py.
- Architecture Fit: AI vision processor.

24. backend/ai_personalize.py

- Purpose: Plan generator.

- Why it exists: Exposes the `/api/ai/personalize` endpoint on port 8000, which processes user profiles and workout data to generate weekly meal plans.
- Who calls it: Express Gateway Route (`backend/routes/generate.js`).
- Exports: APIRouter instance (`router`).
- Data Flow: Accepts incoming JSON payloads, converts them to text string formats, calls the Gemini model, validates the returned JSON structures, and returns the parsed meal plan.
- Interactions: Uses custom system prompts (`SYSTEM_PROMPT`) to get structured JSON outputs from Gemini.
- Dependents: `main.py`.
- Architecture Fit: AI planning pipeline.

25. `backend/shopping_ai.py`

- Purpose: Grocery list parser.
- Why it exists: Exposes the `/api/shopping/generate` endpoint on port 8000, which extracts grocery shopping lists from weekly meal plans.
- Who calls it: Alternate frontend components.
- Exports: APIRouter instance (`router`).
- Data Flow: Accepts weekly plan objects, extracts the ingredients, queries Gemini, parses the response as a JSON array, and returns the result.
- Interactions: Interacts with the Gemini model using specific prompt guidelines.
- Dependents: `main.py`.
- Architecture Fit: AI list extraction processor.

26. `backend/prompts.py`

- Purpose: AI instruction configuration.
- Why it exists: Centralizes prompt structures (like `FOOD_ANALYSIS_PROMPT` and `CHAT_SYSTEM_PROMPT`) to ensure consistent AI outputs.
- Who calls it: `main.py` and `meal_scan.py`.
- Exports: System prompt constants.
- Interactions: Formats raw text strings to define the AI's response formatting and constraints.
- Dependents: `main.py` and `meal_scan.py`.
- Architecture Fit: System configurations template.

27. `frontend/src/main.jsx`

- Purpose: React application mount point.
- Why it exists: Mounts the main React component (`App.jsx`) to the target DOM element in the HTML page.
- Who calls it: Loaded by `index.html` in the browser.
- Exports: None. Renders the application.
- Interactions: Imports the global design system (`index.css`) and renders the `App` component.
- Dependents: Vite bundler.

- Architecture Fit: Frontend entry point.

28. frontend/src/App.jsx

- Purpose: Client router and state manager.
- Why it exists: Coordinates layout rendering, page routing, and fetches active profiles and meal plans from Express during initialization.
- Who calls it: Renders inside main.jsx.
- Exports: App component.
- Data Flow: Fetches state parameters (profile, activePlan, shoppingItems, checkedItems) and distributes them as props to active sub-pages.
- Interactions: Queries the Express backend (/api/profile and /api/generate) and wraps the UI with layout navigation components.
- Dependents: main.jsx.
- Architecture Fit: Client root router.

29. frontend/src/data/constants.js

- Purpose: Static configurations file.
- Why it exists: Holds client-side static lookup values, including search query formats, weekly calendars, and Quick-Commerce links.
- Who calls it: Imported by components such as Dashboard.jsx, MealPlanner.jsx, and Shopping.jsx.
- Exports: DAYS, MEAL_TYPES, ITEM_COSTS, getItemCost, PLATFORMS, and AI_SUGGESTIONS.
- Data Flow: Resolves approximate ingredient prices and formats links for platforms like Blinkit, Zepto, and Instamart.
- Dependents: Various frontend pages.
- Architecture Fit: Client-side data store.

30. frontend/src/hooks/useToast.jsx

- Purpose: Global notifications manager.
- Why it exists: Provides an easy way to trigger toast alerts across different components.
- Who calls it: Pages needing to trigger UI notifications.
- Exports: ToastProvider and the useToast custom hook.
- Data Flow: Appends alert notifications to a state array and automatically clears them after a set delay.
- Dependents: Various page components.
- Architecture Fit: Global notification service.

31. frontend/src/components/layout/Sidebar.jsx

- Purpose: Navigation sidebar.
- Why it exists: Renders navigation links, highlights active routes, and handles toggling the sidebar's collapse state.
- Who calls it: App layout wrapper in App.jsx.
- Exports: Sidebar component.

- Data Flow: Receives `currentPage` and `navigateTo` route dispatchers as props.
- Interactions: Renders navigation links using Lucide Icons.
- Dependents: `App.jsx`.
- Architecture Fit: Navigation UI.

32. frontend/src/components/layout/MobileNav.jsx

- Purpose: Mobile navigation bar.
- Why it exists: Renders a bottom navigation menu for smaller viewports.
- Who calls it: App layout wrapper in `App.jsx`.
- Exports: `MobileNav` component.
- Dependents: `App.jsx`.
- Architecture Fit: Navigation UI.

33. frontend/src/components/charts/WeeklyChart.jsx

- Purpose: Calorie progress visualization.
- Why it exists: Renders a bar chart comparing daily calorie counts against target goal metrics.
- Who calls it: `Dashboard.jsx`.
- Exports: `WeeklyChart` component.
- Data Flow: Maps input values (`day`, `value`, `target`) to percentage-based CSS heights.
- Dependents: `Dashboard.jsx`.
- Architecture Fit: Data visualization.

34. frontend/src/components/charts/MacroRing.jsx

- Purpose: Macronutrient visualization.
- Why it exists: Renders progress rings for tracking proteins, fats, carbs, and calories.
- Who calls it: `FoodScanner.jsx` and other status cards.
- Exports: `MacroRing` component.
- Data Flow: Calculates stroke layouts dynamically using the circle's radius and circumference math:

$\text{Circumference} = 2 * \pi * \text{Radius}$

- Dependents: `FoodScanner.jsx`.
- Architecture Fit: Data visualization.

35. frontend/src/pages/Dashboard.jsx

- Purpose: Main dashboard page.
- Why it exists: Renders the main user dashboard. Displays logged macros, hydration levels, quick navigation actions, and recovery recommendations.
- Who calls it: Route switch statement in `App.jsx`.

- Exports: Dashboard component.
- Data Flow: Calculates daily consumed calories, tracks water intake state, and renders weekly progress charts.
- Interactions: Fetches meal logs from Express (/api/meallog) and updates the state of hydration trackers.
- Dependents: App.jsx.
- Architecture Fit: Core page view.

36. frontend/src/pages/FoodLog.jsx

- Purpose: Meal history log page.
- Why it exists: Displays today's logged meals (divided into breakfast, lunch, dinner, and snacks) and lets users delete or duplicate logs.
- Who calls it: Route switch statement in App.jsx.
- Exports: FoodLog component.
- Data Flow: Fetches database records from /api/meallog, calculates calorie sums per meal type, and handles duplicate operations.
- Interactions: Interacts with the Express server.
- Dependents: App.jsx.
- Architecture Fit: Log page view.

37. frontend/src/pages/FoodScanner.jsx

- Purpose: Food scanner page.
- Why it exists: Allows users to upload food photos, triggers the FastAPI image analysis router, and saves the detected nutrients.
- Who calls it: Route switch statement in App.jsx.
- Exports: FoodScanner component.
- Data Flow: Sends file upload payloads to the FastAPI backend, gets the parsed nutrition details, and saves them to the Express backend (/api/meallog).
- Interactions: Connects with both backend servers.
- Dependents: App.jsx.
- Architecture Fit: Scanner page view.

38. frontend/src/pages/Goals.jsx

- Purpose: Goal configuration page.
- Why it exists: Allows users to select their target weight, diet preferences, water goals, and sleep targets.
- Who calls it: Route switch statement in App.jsx.
- Exports: Goals component.
- Data Flow: Merges user profile data with form inputs and saves the updated goals to the database.
- Interactions: Sends update requests to /api/profile.
- Dependents: App.jsx.

- Architecture Fit: Target settings page view.

39. frontend/src/pages/MealPlanner.jsx

- Purpose: Meal planner grid.
- Why it exists: Renders a weekly meal calendar and provides options to ask the AI for recipes or select replacement suggestions.
- Who calls it: Route switch statement in App.jsx.
- Exports: MealPlanner component.
- Data Flow: Modifies nested cells inside the meal plan state, handles save operations, and loads recipe instructions from the AI chatbot backend on port 8000.
- Interactions: Interfaces with Express (/api/generate) and FastAPI (/api/chat).
- Dependents: App.jsx.
- Architecture Fit: Weekly plan planner.

40. frontend/src/pages/Nutrition.jsx

- Purpose: Nutrition dashboard.
- Why it exists: Displays detailed charts and historical progress metrics for tracked macronutrients.
- Who calls it: Route switch statement in App.jsx.
- Exports: Nutrition component.
- Dependents: App.jsx.
- Architecture Fit: Analysis page view.

41. frontend/src/pages/Onboarding.jsx

- Purpose: Onboarding form.
- Why it exists: Guides new users through a step-by-step onboarding wizard to initialize their profile.
- Who calls it: Render engine inside App.jsx when no active profile is found.
- Exports: Onboarding component.
- Data Flow: Collects user information across form panels, formats commas to list arrays, sends the payload to /api/profile, and triggers a reload of the parent page.
- Interactions: Communicates with the Express backend.
- Dependents: App.jsx.
- Architecture Fit: Initial setup page.

42. frontend/src/pages/Settings.jsx

- Purpose: Settings configuration page.
- Why it exists: Lets users update names, customize system notification preferences, toggle light/dark themes, and trigger new AI weekly plans.
- Who calls it: Route switch statement in App.jsx.
- Exports: Settings component.

- Data Flow: Sends API triggers to start weekly plan generations and updates local dark mode styling.
- Interactions: Connects with Express (/api/generate and /api/profile).
- Dependents: App.jsx.
- Architecture Fit: App configuration page.

43. frontend/src/pages/Shopping.jsx

- Purpose: Grocery shopping page.
- Why it exists: Displays lists of required recipe ingredients and provides Quick-Commerce purchase links.
- Who calls it: Route switch statement in App.jsx.
- Exports: Shopping component.
- Data Flow: Filter items using name search queries, handles check item state updates, and estimates grocery totals.
- Interactions: Opens external search pages for delivery services (Blinkit, Zepto, Swiggy Instamart).
- Dependents: App.jsx.
- Architecture Fit: Grocery page view.

44. frontend/src/pages/WorkoutLog.jsx

- Purpose: Workout log page.
- Why it exists: Lets users log completed workouts (exercise types, active durations, calories burned).
- Who calls it: Route switch statement in App.jsx.
- Exports: WorkoutLog component.
- Data Flow: Submits workout details to the database and reloads logged workouts list.
- Interactions: Communicates with the Express backend.
- Dependents: App.jsx.
- Architecture Fit: Fitness log page view.

PART 4 — API DOCUMENTATION

Express Backend Endpoints (Port 5000)

1. Profile Retrieval

- Route: GET /api/profile
- Purpose: Fetches the user profile (used to determine if onboarding is complete).
- Request Body: None.
- Response:

```
{ "_id": "60d5ec49", "name": "Niharika", "age": 22, "gender": "female", "height": 165, "currentWeight": 58,
"targetWeight": 54, "primaryGoal": "Lose Fat", "dietType": "balanced", "allergies": ["Peanuts"], "foodsToAvoid":
["Soy"], "waterGoal": 3, "sleepHours": 8 }
```

- Validation: Verifies if database entries exist; returns a 404 status code if no profile is found.
- MongoDB Collection: userprofiles.
- Frontend Caller: App.jsx (loaded in loadData() during startup).
- Controller: Express Router wrapper.
- Model: UserProfile.js.

2. Profile Submission

- Route: POST /api/profile
- Purpose: Saves onboarding profile data to the database.
- Request Body: JSON mapping of demographic properties.
- Response: The saved UserProfile document with status code 201.
- Validation: Schema-level validations (names, ages, heights, etc., are flagged as required).
- MongoDB Collection: userprofiles.
- Frontend Caller: Onboarding.jsx / Goals.jsx.
- Model: UserProfile.js.

3. Logged Workouts Fetch

- Route: GET /api/workouts
- Purpose: Returns logged workouts, sorted from newest to oldest.
- Request Body: None.
- Response:

```
[ { "_id": "60d5ec92", "type": "Running", "duration": 30, "caloriesBurned": 350, "exercises": ["Outdoor jog"],
  "date": "2026-07-11T00:00:00.000Z" } ]
```

- MongoDB Collection: workoutlogs.
- Frontend Caller: WorkoutLog.jsx.
- Model: WorkoutLog.js.

4. Active Plan Save Router

- Route: POST /api/generate
 - Purpose: Intermediary endpoint that calls FastAPI, retrieves the generated plan, and saves it.
 - Request Body: None.
 - Response: The newly saved MealPlan document.
 - Validation: Checks if a valid user profile exists before calling the generation API.
 - MongoDB Collection: mealplans.
 - Frontend Caller: Settings.jsx (triggered when requesting a new AI plan).
 - Model: MealPlan.js (references UserProfile).
-

FastAPI Backend Endpoints (Port 8000)

1. Meal Scanner Route

- Route: POST /api/meals/scan
- Purpose: Receives food photos, queries the Gemini model, and returns nutrition estimates.
- Request Body: multipart/form-data with an image file key named "image".
- Response:

```
{ "food_name": "Avocado Toast with Egg", "calories": 350, "carbs": 24, "protein": 14, "fats": 18, "image_url": "http://localhost:5000/uploads/temp.jpg" }
```

- Validation: Validates image inputs using Pydantic models.
- MongoDB Collection: None (handled on port 5000).
- Frontend Caller: FoodScanner.jsx (triggered when analyzing uploads).
- Controller: meal_scan.py.

2. Chat Support Route

- Route: POST /api/chat
- Purpose: Processes conversation messages and returns response text.
- Request Body:

```
{ "messages": [ { "role": "user", "content": "What are some healthy fat options?" } ] }
```

- Response:

```
{ "response": "Healthy fats include avocados, olive oil, almonds, walnuts, and chia seeds." }
```

- Validation: Validates payload structure using Pydantic schemas.
- Frontend Caller: AICoach.jsx (chat window) / MealPlanner.jsx (recipe finder).
- Controller: main.py (chat handler).

3. AI Plan Personalization Route

- Route: POST /api/ai/personalize
- Purpose: Generates weekly recipes and grocery lists based on profile parameters.
- Request Body: Contains profile parameters and workout arrays.
- Response: Conforms to the nested weekly plan JSON schema.
- Validation: Validates input data with Pydantic typing definitions.
- Frontend Caller: Express Server (routes/generate.js).
- Controller: ai_personalize.py.

PART 5 — COMPLETE FEATURE FLOWS

For food photo analysis, images are sent as dictionaries containing the MIME type and base64-encoded byte string:

```
img_part = {'mime_type': image.content_type, 'data': img_data} response = model.generate_content([img_part, "Prompt Instructions"])
```

- Chat Support Sessions:

For chatbot interactions, the system formats chat history by converting message roles and starts a conversation session to maintain context:

```
chat_session = model.start_chat(history=formatted_history) response = chat_session.send_message(user_input_string)
```

The chat session automatically manages state by appending new messages to the active conversation history.

PART 7 — FASTAPI ARCHITECTURE

1. Why FastAPI?

1. Asynchronous Request Handling: Features an event loop that processes multiple incoming requests concurrently without blocking processes.
2. Pydantic Validation: Automatically parses JSON payloads and returns detailed validation errors if structure types mismatch.
3. Speed: Built on top of Starlette and Uvicorn, offering performance on par with Go and Node.js.

2. Request Validation Example

```
class ChatMessage(BaseModel): role: str # Checks if the role string exists content: str # Checks if the content string exists class ChatRequest(BaseModel): messages: List[ChatMessage] # Enforces array lists of objects matching ChatMessage
```

If a payload contains invalid datatypes (e.g., passing numbers instead of string message text), FastAPI returns a 422 Unprocessable Entity error automatically.

PART 8 — EXPRESS CONFIGURATION

CRUD Operations Routing in Express

The Express server handles all core database transactions. It manages writes, edits, deletions, and queries for logged meals, profile updates, and active fitness goals, allowing the FastAPI server to focus entirely on running AI tasks.

PART 9 — DATABASE STRUCTURE (MONGODB)

Connecting to MongoDB Atlas

Connection retry logic inside backend/config/db.js:

```
async function connectWithRetry(delay = 1000) { try { await mongoose.connect(mongoURI); console.log("■ Connected to MongoDB"); } catch (err) { console.error("■ MongoDB connection failed:", err.message); setTimeout(() => { connectWithRetry(Math.min(delay * 2, 5000)); }, delay); } }
```

If a network timeout occurs, this function retries the database connection, doubling the delay each time up to a maximum of 5 seconds.

PART 10 — USER AUTHENTICATION BLUEPRINT

Currently, BiteBuddy operates in a sandbox development mode using a static database entry ("demo-user"). Here is a blueprint to implement secure user authentication:

1. Schema Extensions (backend/models/User.js)

```
const mongoose = require('mongoose'); const bcrypt = require('bcryptjs'); const userSchema = new mongoose.Schema({ email: { type: String, required: true, unique: true, lowercase: true }, password: { type: String, required: true }, }, { timestamps: true }); // Pre-save hook: Hash passwords before saving to the database userSchema.pre('save', async function(next) { if (!this.isModified('password')) return next(); this.password = await bcrypt.hash(this.password, 12); next(); }); module.exports = mongoose.model('User', userSchema);
```

2. Authorization Flow

1. Signup/Login: The server validates user credentials, checks password hashes using `bcrypt.compare()`, generates a signed JSON Web Token (JWT), and returns it to the client.
2. Access Token Verification: The client stores the JWT in secure, HTTP-only cookies.
3. Middleware Validation: An Express middleware (`requireAuth.js`) intercepts incoming API requests, extracts the JWT, verifies the signature, and sets the authenticated user's ID on the request object (`req.userId`).

PART 11 — DATA FLOW DIAGRAMS

1. Goal Settings Persistence

```
[User enters details in Goals.jsx] ➡ [Triggers fetch POST to /api/profile] ■ ▼ [Express Server] ■■■■■■■■> [Runs UserProfile.findOneAndUpdate() upsert] ▲ ■ ■ ▼ [Refreshes Page state] <■ [Returns updated Profile document to client]
```

2. Generation of AI Meal Plan

```
[Settings.jsx triggers Plan Generation] ➡ [POST sent to Express /api/generate] ■ ▼ [Express gathers UserProfile & WorkoutLogs] ■■■■■■■■> [POSTs to FastAPI /api/ai/personalize] ■ ▼ [FastAPI queries Gemini and validates JSON] ■■■■■■■■> [Returns weekly plan payload] ■ ▼ [Express saves plan using new MealPlan()] ■■■■> [Saves to MongoDB and returns to React]
```

PART 12 — FRONTEND SYSTEM (REACT)

State Management & Props Flow

BiteBuddy uses React's state lifting pattern to manage global application data from the root component (App.jsx):

- Props Distribution: Global states (such as profile, activePlan, and shoppingItems) are lifted to App.jsx and passed down as read-only properties to pages like Dashboard and MealPlanner.
- Callback Triggers: Sub-components trigger state updates using callback functions passed down from App.jsx (such as navigateTo and setShoppingItems).

Core React Hooks Used

1. useState: Manages component-level state changes (such as tracking input fields, loading states, and open modals).
 2. useEffect: Triggers side effects, like fetching active profiles and meal plans from the backend when the app first loads.
 3. useCallback: Memoizes functions to prevent unnecessary re-renders. For example, in AICoach.jsx, sendMessage is memoized to avoid creating a new function reference on every render.
-

PART 13 — 100 VIVA & INTERVIEW QUESTIONS

1. What does the acronym MERN stand for?

MERN stands for MongoDB, Express.js, React.js, and Node.js. It is a full-stack JavaScript development framework.

2. What is the role of the FastAPI server in this project?

It acts as a dedicated gateway for AI processing. It connects to the Google Generative AI SDK, processes uploaded images, and parses text outputs into JSON formats.

3. What port is the Express server listening on?

Port 5000 is the default backend port.

4. Which port handles the FastAPI requests?

Port 8000.

5. How does the React frontend run during local development?

It runs on port 5173 using the Vite development server.

6. What is MongoDB?

MongoDB is a document-oriented, NoSQL database that stores data in flexible, JSON-like BSON documents.

7. What is Mongoose?

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It provides schema validation and manages relationships between data models.

8. What is Express?

Express is a minimal and flexible web application framework for Node.js, used to build APIs and handle HTTP requests.

9. What is React?

React is a declarative JavaScript library developed by Meta, used for building component-based user interfaces.

10. Why is uvicorn used in this project?

Uvicorn is an ASGI (Asynchronous Server Gateway Interface) web server implementation for Python, used to run the FastAPI application.

11. What model name is defined in your Gemini calls?

gemini-2.5-flash.

12. What does CORS stand for, and why is it configured?

CORS stands for Cross-Origin Resource Sharing. It is configured to allow the React frontend (running on port 5173) to securely request resources from the backend servers (ports 5000 and 8000).

13. How are node packages managed on the Express server?

Using the npm package manager and the dependencies listed in package.json.

14. What are python dependencies listed under?

backend/requirement.txt.

15. Where is the MongoDB connection URI stored?

In the environment variables configuration file (backend/.env).

16. What does dotenv do?

It loads environment variables from a .env file into process.env in Node.js, and os.environ in Python.

17. What does the onboarding step check for?

It checks if the database contains a UserProfile document. If not, it redirects the user to the onboarding wizard.

18. How do you view uploaded images?

Images are stored in backend/uploads/ and served statically via the Express path /uploads/:filename.

19. What does the strictQuery setting do in db.js?

```
mongoose.set("strictQuery", true);
```

It ensures that fields not defined in the schema are ignored when querying the database.

20. What is a schema in Mongoose?

A schema defines the structure of documents within a MongoDB collection, including fields, types, default values, and validations.

PART 14 — CODE EXPLANATIONS

1. backend/main.py

This file serves as the entry point for the Python FastAPI server.

```
from fastapi import FastAPI, HTTPException from fastapi.middleware.cors import CORSMiddleware from pydantic import BaseModel from typing import List import os import google.generativeai as genai from prompts import CHAT_SYSTEM_PROMPT api_key = os.getenv("GEMINIKEY") genai.configure(api_key=api_key) model = genai.GenerativeModel(model_name="gemini-2.5-flash", system_instruction=CHAT_SYSTEM_PROMPT ) app = FastAPI(title="BiteBuddy API") app.add_middleware( CORSMiddleware, allow_origins=["*"], allow_credentials=True, allow_methods=["*"], allow_headers=["*"], )
```

PART 15 — ARCHITECTURE ANALYSIS & IMPROVEMENTS

1. File Uploads

- Current Limitation: Storing uploaded files locally on the backend server does not scale and risks data loss if the server instance restarts.
- Startup Solution: Stream uploads directly to cloud storage (like Amazon S3 or Google Cloud Storage) and save the CDN references in the database.

2. Server Architecture

- Current Limitation: Running Node and Python backend services as independent local processes is difficult to scale and manage.
- Startup Solution: Containerize both services using Docker, deploy them to orchestrators like Kubernetes, and place them behind API Gateways to handle load balancing and routing.